

Back Propagation Algorithm

Prof G C Nandi

Setup of the Network

- Let us consider a **3-layer neural network**:

- **Input layer**: nodes indexed by i , inputs x_i .
- **Hidden layer**: nodes indexed by j .
- **Output layer**: nodes indexed by k .

Parameters:

- w_{ji} : weight from input node i to hidden node j .
- w_{kj} : weight from hidden node j to output node k .
- Bias: w_{j0}, w_{k0} .

Activation functions:

- Hidden: $h_j = f(z_j)$, with $z_j = \sum_i w_{ji}x_i + w_{j0}$.
- Output: $\hat{y}_k = f(z_k)$, with $z_k = \sum_j w_{kj}h_j + w_{k0}$.

Error (loss for sample m):

$$e^{(m)} = \frac{1}{2} \sum_k (Y_k^{(m)} - \hat{y}_k^{(m)})^2$$

Role of the Chain Rule in Backpropagation

Role of the Chain Rule in Backpropagation

We want to **adjust weights** so that error $e^{(m)}$ decreases.

That means computing gradient:

$$\frac{\partial e^{(m)}}{\partial w}$$

Here comes the **chain rule**: each weight indirectly affects error through multiple layers.

Gradient at Output Layer

Take a weight from hidden to output: w_{kj} .

$$\frac{\partial e^{(m)}}{\partial w_{kj}} = \frac{\partial e^{(m)}}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}}$$

- $\frac{\partial e}{\partial \hat{y}_k} = -(Y_k - \hat{y}_k)$
- $\frac{\partial \hat{y}_k}{\partial z_k} = f'(z_k)$
- $\frac{\partial z_k}{\partial w_{kj}} = h_j$

So:

$$\frac{\partial e^{(m)}}{\partial w_{kj}} = \delta_k \cdot h_j$$

where we define the **error signal** at output node k :

$$\delta_k = (\hat{y}_k - Y_k) f'(z_k)$$

Notice how **error flows backwards**

- Output error δ_k influences hidden layer through weights w_{kj} .
- Then hidden error δ_j propagates further to input connections.

Gradient at Hidden Layer

Now for weight from input to hidden: w_{ji} .

$$\frac{\partial e^{(m)}}{\partial w_{ji}} = \frac{\partial e^{(m)}}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}}$$

But $\frac{\partial e}{\partial h_j}$ comes from **all outputs connected to hidden unit j** :

$$\frac{\partial e}{\partial h_j} = \sum_k \frac{\partial e}{\partial z_k} \cdot \frac{\partial z_k}{\partial h_j} = \sum_k \delta_k \cdot w_{kj}$$

Thus,

$$\frac{\partial e^{(m)}}{\partial w_{ji}} = \delta_j \cdot x_i$$

where we define the **error signal** at hidden node j :

$$\delta_j = f'(z_j) \cdot \sum_k w_{kj} \delta_k$$

Notice how **error flows backwards**:

- Output error δ_k influences hidden layer through weights w_{kj} .
- Then hidden error δ_j propagates further to input connections.

Weight Update Rule

For learning rate η :

$$w \leftarrow w - \eta \frac{\partial e}{\partial w}$$

Explicitly:

- Output weights:

$$w_{kj} \leftarrow w_{kj} - \eta \delta_k h_j$$

- Hidden weights:

$$w_{ji} \leftarrow w_{ji} - \eta \delta_j x_i$$

Bias terms follow similarly with multiplier = 1.

Class Assignment

Write down the Backpropagation algorithm for solving the following nonlinear problem:

Train a 2-input X-OR by a neural network having two non computing neurons at the input layer, followed by 2 hidden layers with 2 computing neurons each layer and finally having one output layer having one computing neuron. Use: 1) sigmoid activation function for

all computing neurons and 2) use ReLU activation for hidden layers and sigmoid activation function in the output layer.

Compare the training performance for the both.

Compute weight updates for one epoch by hand calculations and rest train by creating a program.